

SearchClaw: Towards Building Powerful Search Agent using Harness Engineering

Yutao Zhu✉, Jiajie Jin, Zhicheng Dou

Affiliation: Gaoling School of Artificial Intelligence, Renmin University of China

Large language models (LLMs) have demonstrated remarkable capabilities in reasoning, planning, and tool use, yet building effective autonomous agents for deep web research remains an open challenge. Existing approaches typically address this problem through either training specialized smaller models for search-related subtasks or designing complex workflow frameworks, resulting in fragmented systems that fail to fully leverage the emergent abilities of powerful foundation models. Recently, the striking success of LLM-powered coding agents has drawn increasing attention to the paradigm of *harness engineering*, which focuses on the systematic design of external control infrastructure surrounding a strong base model, including tool orchestration, state management, and quality enforcement mechanisms, rather than modifying the model itself. In this paper, we explore whether this paradigm can be effectively transferred to the domain of deep web research. We present **SearchClaw**, a deep research agent built around a unified agentic loop that tightly integrates reasoning, retrieval, and quality verification within a single harness. The system features structured research planning for query decomposition and progress tracking, pre-injection content compression for efficient context window management, and hook-based quality gates that enforce source diversity and coverage requirements before answer finalization. To our knowledge, SearchClaw represents one of the first systematic attempts to apply harness engineering principles to build a capable deep search agent, and we hope this exploration provides useful insights for future work in this direction.

Note: This is a living technical report documenting an ongoing project. Some design components described herein are still under active development, and implementation details may evolve as we iterate on the system. We intend to update this report periodically to reflect the latest architectural changes and to incorporate experimental results as they become available.

✉ **Contact:** yutaozhu94@gmail.com

🔗 **Code:** <https://github.com/RUC-NLPIR/SearchClaw>



**Gaoling School of
Artificial Intelligence**

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in reasoning, planning, and tool use, enabling a new generation of autonomous agents that can interact with external environments to accomplish complex tasks (Schick et al., 2023; Shen et al., 2023; Yao et al., 2023; Qin et al., 2024). Among the most challenging applications for such agents is deep web research, where the goal is to answer complex, open-ended user queries by autonomously searching the internet, reading and comprehending large volumes of web content, cross-verifying information across multiple sources, and synthesizing the findings into a well-structured, citation-grounded response or report (Li et al., 2025d; Zheng et al., 2025; Li et al., 2025a). Different from single-turn question answering, deep web research requires a combination of various capabilities: the agent must dynamically formulate and reformulate search strategies as new information emerges, manage a limited context window while processing potentially dozens of lengthy web pages, maintain coherent tracking of which aspects of a multi-faceted query have been addressed, and enforce quality standards on the final output to ensure factual accuracy, source diversity, and comprehensive coverage. These requirements make deep web research a particularly demanding testbed for the design of LLM-based autonomous agents, and a setting where naive approaches that simply prompt a powerful model with a search tool quickly reach their limits.

Existing efforts toward building deep search agents can be broadly categorized into two groups: (1) The first focuses on designing agentic frameworks or pipelines that orchestrate the search process, integrating search, retrieval, and content

refinement into the reasoning process of LLMs through various workflow designs (Li et al., 2025c,d; Zheng et al., 2025). While these systems have demonstrated the feasibility of LLM-powered deep research, their architectures are typically coupled with specific workflow patterns and do not explicitly consider the systematic design of the external control infrastructure surrounding the base model. (2) The second group of research explores training open-source models to internalize search and retrieval capabilities through data synthesis, supervised fine-tuning, and reinforcement learning (Jin et al., 2025; Song et al., 2025; Wu et al., 2025b; Li et al., 2025b,a). Although these training-based approaches have achieved impressive results on multiple benchmarks, they require substantial resources for data construction and model training, and their effectiveness remains fundamentally bounded by the base model’s intrinsic reasoning capacity. Despite the progress made by both lines of work, neither has systematically explored how to maximize the performance of a single powerful foundation model on deep search tasks through principled design of the surrounding control infrastructure.

Recently, the striking success of LLM-powered coding agents has drawn significant attention to a paradigm that offers a promising alternative. Systems such as Claude Code and Codex have demonstrated that a single powerful foundation model,¹ when equipped with well-engineered external infrastructure, can achieve remarkable performance on complex, long-horizon software engineering tasks without task-specific model training. This paradigm, increasingly referred to as *harness engineering* (Rajasekaran, 2026; Lopopolo, 2026; Lee et al., 2026), shifts the focus of system intelligence from model adaptation to infrastructure design. Rather than modifying model weights or orchestrating multiple specialized agents, harness engineering focuses on building the control layer that surrounds a strong base model, encompassing tool orchestration, state management, context window control, quality enforcement, and permission governance. The core insight is that when the base model already possesses strong reasoning, planning, and tool-use capabilities, the bottleneck to system performance often lies not in the model itself but in the infrastructure that mediates its interaction with the external environment. While this insight has been extensively validated in the coding domain, it has not yet been systematically applied to the construction of deep search agents, where the challenges of open-ended information sources, noisy web content, and multi-dimensional quality assessment present a distinct set of infrastructure design problems.

In this paper, we explore whether the harness engineering paradigm can be effectively transferred from coding to deep web research, and present **SearchClaw** as an initial attempt in this direction. SearchClaw is built around a single agentic loop where one powerful foundation model drives the entire research process through a continuous cycle of reasoning, tool use, and quality verification. Instead of distributing intelligence across multiple specialized agents or training a dedicated model, we concentrate our engineering effort on the harness layer that shapes how the model perceives, plans, acts, and self-corrects. Concretely, the harness provides four key mechanisms: (1) a structured research planning module that decomposes complex queries into tracked sub-tasks, preventing the well-known shallow-coverage failure mode where the model thoroughly investigates one aspect of a query while neglecting others; (2) a pre-injection content compression pipeline that employs LLM-based extraction to reduce raw web page content before it enters the main context window, enabling the model to consult significantly more sources per session; (3) a hook-based quality gate system that evaluates citation diversity, source coverage, and plan completeness at each potential stopping point, injecting targeted feedback to force continued research when standards are not met; and (4) a persistent cross-session memory system that carries user preferences and source reputation across independent research sessions. We note that the deep search setting introduces harness design challenges that are largely absent in coding: information sources are open-ended and uncontrollable in quality, raw web content is orders of magnitude more verbose than source code files, and answer quality must be assessed along multiple dimensions including factual grounding, source diversity, and topical coverage, none of which admit the kind of deterministic verification (such as compilation or test execution) available in coding environments.

To summarize, the main contributions of this paper are as follows:

- We propose applying the harness engineering paradigm, which has proven effective in autonomous coding agents, to the construction of deep web research agents. We argue that when a sufficiently powerful foundation model is available, systematic design of the surrounding control infrastructure can be more practical and effective than training specialized models or engineering complex multi-agent workflows.
- We design and implement SearchClaw, a deep research agent built around a single agentic loop architecture. The system incorporates several key harness components including structured research planning, pre-injection

¹Claude Code: <https://code.claude.com/>, OpenAI Codex: <https://openai.com/codex/>.

content compression, hook-based quality gates, and persistent cross-session memory, each addressing a specific challenge in the deep search setting.

- We provide a detailed discussion of the design principles, engineering trade-offs, and domain-specific adaptations involved in transferring harness engineering from coding to deep web research, offering practical insights for future work on harness-centric agent construction in other domains.

2 Related Work

2.1 Deep Research Agent

The idea of augmenting language models with web search capabilities can be traced back to WebGPT (?), which trained a model to interact with a text-based web browser via human feedback, and the ReAct framework (Yao et al., 2023), which established the foundational paradigm of interleaving reasoning with action execution for tool-augmented LLM agents.

The emergence of large reasoning models such as OpenAI o1 and DeepSeek-R1 (Guo et al., 2025), has catalyzed a new generation of deep search agents. One prominent direction designs agentic frameworks that integrate retrieval directly into extended reasoning processes, enabling models to dynamically search, navigate web pages, refine retrieved content, and synthesize findings within a unified workflow (Li et al., 2025c,d; Zheng et al., 2025). These systems have demonstrated the feasibility of autonomous deep research and pushed performance on knowledge-intensive benchmarks, though their architectures are generally coupled with specific workflow patterns or reasoning models.

A parallel direction focuses on training open-source models to internalize search capabilities through reinforcement learning. Several works explore outcome-based RL approaches where models learn to autonomously generate search queries during step-by-step reasoning, with techniques such as retrieved token masking, curriculum-based training, and simulated search environments to improve training stability and reduce API costs (Jin et al., 2025; Song et al., 2025; Sun et al., 2025). The most comprehensive effort in this direction comes from Tongyi Lab, which has developed a systematic training methodology through an extensive family of interconnected works spanning benchmark construction, trajectory synthesis, data formalization, and scalable reinforcement learning (Wu et al., 2025b,a; Li et al., 2025b; Tao et al., 2025), culminating in an end-to-end trained agentic model that achieves state-of-the-art results across multiple deep research benchmarks (Li et al., 2025a).

At the industry level, proprietary deep research systems from major AI companies, including OpenAI Deep Research, Google Gemini Deep Research, Grok DeepSearch, and Perplexity Deep Research, have demonstrated the commercial viability of this capability, though their technical details remain largely undisclosed. Open-source system-level efforts such as DeerFlow and STORM (Shao et al., 2024) provide alternative architectures through multi-agent orchestration and structured report generation pipelines.

Our work differs from all the above in its central thesis. Rather than designing a new workflow, training a specialized model, or building a multi-agent pipeline, SearchClaw explores how principled harness engineering around a single powerful foundation model can yield a capable deep search agent, a perspective that has not been systematically investigated in the deep research domain.

2.2 Harness Engineering

The concept of an agent harness, the external control infrastructure surrounding a language model that mediates its interaction with the environment, has roots in early work on tool-augmented LLMs such as Toolformer (Schick et al., 2023) and general-purpose agent frameworks like LangChain and AutoGen that provide middleware for tool routing, memory management, and multi-step orchestration.

Harness engineering emerged as a distinct discipline primarily through the success of LLM-powered coding agents. Systems such as Claude Code and OpenAI Codex demonstrated that a single powerful model, when embedded within a carefully designed harness encompassing tool dispatch, permission governance, context management, and feedback loops, could achieve remarkable performance on long-horizon software engineering tasks without task-specific model training (Lopopolo, 2026). Anthropic’s engineering team has published a series of studies establishing key harness design principles for long-running agents, including context compaction strategies, multi-context-window continuity

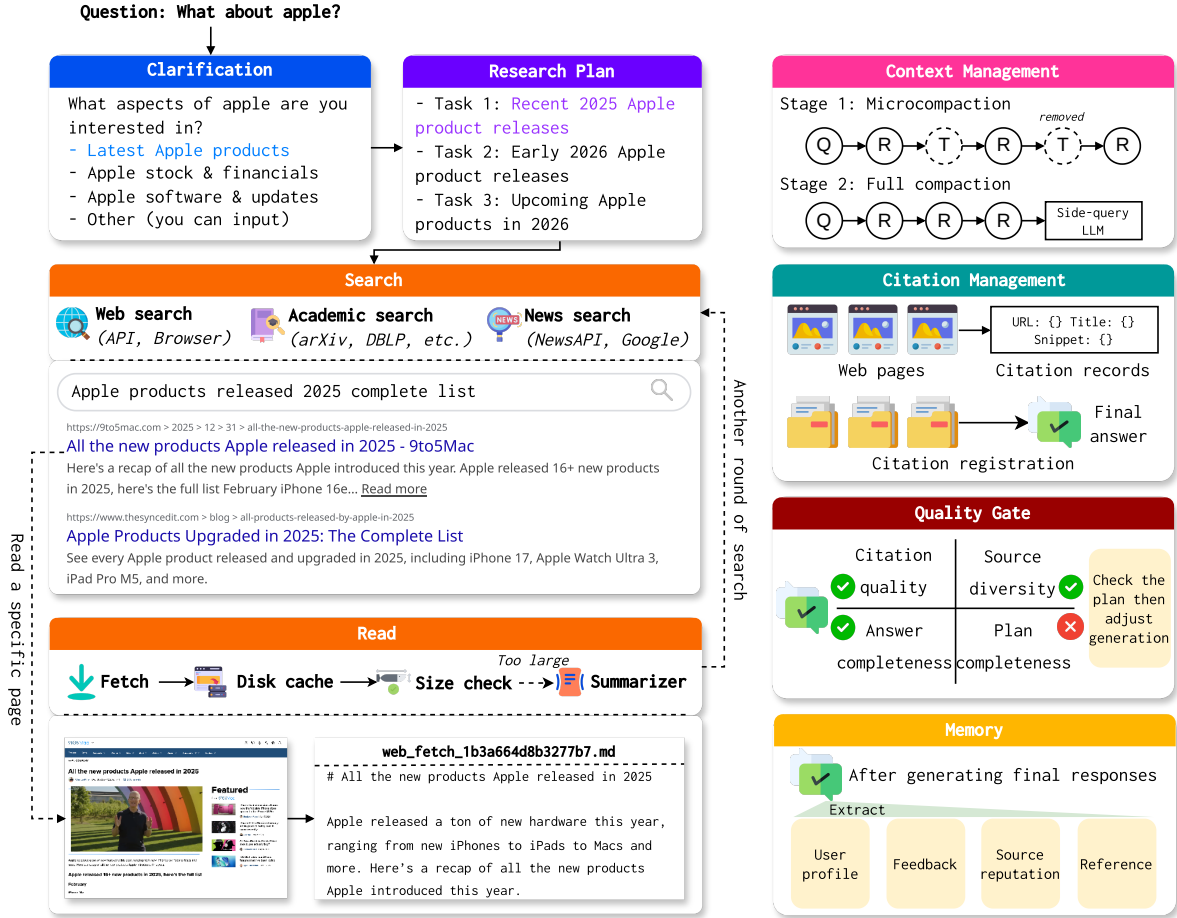


Figure 1 The architecture of SearchClaw.

mechanisms, and evaluator-generator decoupling for self-assessment calibration (Rajasekaran, 2026). Common patterns that have emerged across coding agent harnesses include hook-based lifecycle enforcement, progressive skill disclosure for instruction budget management, and sub-agent delegation for cost-efficient task decomposition. More recently, Meta-Harness (Lee et al., 2026) has explored automating the harness design process itself through an outer-loop search over harness code, demonstrating that harness optimization can surpass hand-engineered baselines and reinforcing the premise that the harness is a critical and optimizable determinant of system performance.

Despite the rapid maturation of harness engineering in the coding domain, its application beyond coding remains largely unexplored. The coding setting benefits from deterministic feedback signals such as compilation errors and test results that provide natural quality gates. Domains like deep web research lack such signals, requiring the harness to construct its own quality assessment mechanisms. SearchClaw represents an attempt to bridge this gap by transferring harness engineering principles to the deep search domain while adapting key components to the distinct challenges of open-ended information retrieval and synthesis.

3 Architecture and Design

3.1 Overview

SearchClaw is designed around a central principle inherited from the harness engineering paradigm: trust the base model’s reasoning and planning capabilities, and focus engineering effort on the infrastructure that shapes how the model perceives information, executes actions, and verifies its own output. The system architecture reflects this principle through a single agentic loop that serves as the unified control flow for the entire research process.

Algorithm 1: SearchClaw Agentic Research Loop

Input: Query q , tool registry $\mathcal{T}$, hook engine $\mathcal{H}$, memory store $\mathcal{M}$, max turns T , compaction threshold θ **Output:** Answer a with citation set $\mathcal{C}$

```
1  $prompt \leftarrow \text{BUILDPROMPT}(\mathcal{T}, \text{RETRIEVEMEMORIES}(\mathcal{M}, q));$ 
2  $msgs \leftarrow [\text{USERMSG}(q)]; \mathcal{C} \leftarrow \emptyset; plan \leftarrow \text{null}; t \leftarrow 0;$ 
3 while true do
4    $t \leftarrow t + 1;$ 
5   if  $t > T$  then
6      $a \leftarrow \text{FORCESYNTHESIS}(msgs, prompt);$  break; // Guard: force synthesis if max turns exceeded
7   end if
8   if  $\text{TOKENS}(msgs) > \theta$  then // Context management
9      $msgs \leftarrow \text{COMPACT}(msgs, \theta);$ 
10  end if
11   $\langle text, tools \rangle \leftarrow \text{LLM}(msgs, prompt, \mathcal{T});$ 
12  Append assistant message to  $msgs$ ;
13  if  $tools = \emptyset$  then // Model wants to finalize
14     $\langle pass, fb \rangle \leftarrow \text{QUALITYGATE}(\mathcal{H}, \mathcal{C}, plan, msgs);$ 
15    if  $pass$  then  $a \leftarrow text$ ; break;
16    else Append  $\text{USERMSG}(fb)$  to  $msgs$ ;
17  else
18     $results \leftarrow \text{EXECTOOLS}(tools, \mathcal{T});$  // Parallel when safe
19    foreach  $r_i \in results$  do
20      if  $r_i$  is pending question then // Interactive clarification
21        Yield question to user;  $r_i \leftarrow$  user's answer;
22      else if  $r_i$  is plan update then // Plan management
23         $plan \leftarrow r_i.plan$ ;
24      else if  $r_i.citations \neq \emptyset$  then // Citation registration
25         $\mathcal{C} \leftarrow \mathcal{C} \cup r_i.citations$ ;
26      end if
27      Append  $r_i$  to  $msgs$ ;
28    end foreach
29  end if
30 end while
31  $\text{EXTRACTMEMORIES}(\mathcal{M}, q, a, plan);$  // Memory management
32 return  $a, \mathcal{C};$ 
```

The agentic loop is implemented as an asynchronous iterative process where each iteration follows a six-phase cycle (as illustrated in Algorithm 1). **First**, a guard check determines whether the agent has exceeded the maximum number of allowed turns, forcing a final synthesis if the limit is reached. **Second**, a compaction check evaluates whether the accumulated context has grown beyond a configurable threshold, triggering context compression if necessary (Section 3.7). **Third**, the loop issues an LLM call that returns the model's response, which may contain reasoning text, tool invocations, or both. **Fourth**, a tool routing phase dispatches any requested tool calls for execution, with concurrency-safe tools, such as web search (Section 3.3) and page retrieval (Section 3.4), running in parallel and state-mutating tools (such as research plan updates) running sequentially. **Fifth**, if the model produced a response without requesting any tools, signaling its intent to finalize, a set of quality gates evaluate the current session state against configurable quality criteria (Section 3.6). If any gate fails, targeted feedback is injected into the conversation and the loop continues. **Finally**, tool results, newly discovered citations, and plan updates are injected back into the conversation, and control returns to the first phase.

All mutable state throughout this process, including the conversation history, accumulated citations, turn count, and the current research plan, is carried in a single explicit state object that the loop reads and writes at each iteration. This design eliminates hidden global state and shared mutable singletons, making the loop's behavior deterministic

Table 1 Tool registry of SearchClaw.

Tool	Category	Function
ask_user	Interaction	Ask the user a clarifying question with selectable options.
research_plan	Planning	Create, update, or check a structured research plan with sub-tasks.
web_search	Discovery	Search the web via search engine API and return ranked results.
academic_search	Discovery	Search for academic papers via Semantic Scholar, DBLP, and arXiv.
news_search	Discovery	Search for recent news articles within a configurable time window.
web_fetch	Retrieval	Fetch a web page and convert it to readable markdown.
deep_read	Retrieval	Read a specific section from a previously cached web page.
cite_source	Synthesis	Register a citation that supports a specific claim in the answer.

given a fixed sequence of LLM responses and tool results.

Several design decisions in this architecture merit discussion. The choice of a single loop over a multi-agent pipeline reflects the observation that deep web research is an inherently sequential reasoning process: each search informs which pages to read, each page informs the next query, and the final synthesis depends on all gathered evidence. Distributing this process across multiple agents would introduce inter-agent communication overhead and complicate state management without a clear benefit, since the base model is already capable of handling the full reasoning chain. Parallelism is instead applied at the tool execution level, where independent network requests can safely proceed concurrently. The six-phase structure also embeds two critical safety mechanisms directly into the loop rather than leaving them to external orchestration: context compaction prevents the session from exceeding the model’s context window, and quality gates prevent premature termination with insufficient evidence. By making these mechanisms integral parts of each iteration, the system guarantees that every potential stopping point is evaluated against explicit quality standards.

Figure 1 illustrates the overall architecture of SearchClaw, while Table 1 lists all registered tools. The following sections describe each harness component in detail.

3.2 Research Planning

A well-known failure mode of LLM-based research agents is shallow coverage on multi-faceted queries. Without explicit planning mechanisms, the model tends to investigate the first aspect it encounters in depth, then generate a partial answer that neglects the remaining dimensions. For instance, given a comparative query such as “*Compare the economic policies of the US, EU, and China on AI regulation*”, an unguided agent may thoroughly research the US position but dedicate only superficial treatment to the EU and China. This failure arises not from a lack of reasoning capability but from the absence of structural scaffolding that tracks what has been covered and what remains.

To address this, SearchClaw introduces a structured research planning module that enables the model to decompose a complex query into three to seven tracked sub-tasks before beginning its search. Each sub-task maintains a title, optional strategic details describing the intended investigation approach, a status indicator (pending, in-progress, or completed), and a findings field where the agent records key information as it is discovered. The module exposes three actions through a single unified tool interface: (1) create, which decomposes the query into an initial set of sub-tasks; (2) update, which modifies a sub-task’s status and records findings; and (3) check, which allows the agent to review the current plan state and progress. Consolidating these actions into a single tool keeps the tool namespace compact and reduces the prompt overhead that would result from registering multiple separate tools.

Three complementary mechanisms ensure that the research plan faithfully reflects the user’s intent and that the model follows through on it rather than abandoning it prematurely. The first is an **interactive clarification mechanism**: before committing to a research plan, the agent may pause and present the user with a clarifying question and a set of selectable options when the query is ambiguous or underspecified. This mechanism ensures that the plan is grounded in the user’s actual information need rather than the model’s potentially incorrect interpretation. The decision to trigger clarification is made autonomously by the model, which is instructed to initiate it *only when* disambiguation would meaningfully help determine the research direction, and to skip it entirely when the query is already clear and specific. To minimize disruption to the research flow, clarification is limited to at most one question per session. The second is a **soft nudge**: if the agent has performed three or more searches without creating a plan, the harness injects a

synthetic message suggesting that the query appears multi-faceted and would benefit from structured decomposition. This nudge fires at most once per session to avoid repetitive interference. The third is a **hard enforcement** through the plan completeness hook integrated into the quality gate system (Section 3.6): when the model attempts to finalize its answer, the hook checks whether any sub-tasks remain incomplete and, if so, blocks finalization and injects feedback listing the unfinished topics, forcing the agent to continue its research.

Beyond guiding the research process, the plan’s accumulated findings serve a dual purpose. During the session, recorded findings provide the model with structured awareness of what it has already learned, which is particularly valuable after context compaction (Section 3.7) has removed older tool results from the conversation history. The findings effectively function as a compressed, organized summary of research progress that survives context management operations. After the session concludes, the structured findings are passed to the memory extraction system (Section 3.8), providing high-quality input for identifying insights worth persisting across future sessions.

3.3 Search

A deep research agent’s effectiveness is fundamentally constrained by the breadth and reliability of its information acquisition capabilities. SearchClaw provides the model with multiple discovery tools spanning different information domains: a general web search tool for broad queries, a news search tool with configurable time windows for current-events research, and an optional academic search tool that queries scholarly databases when peer-reviewed sources are required. This multi-source design reflects the observation that different types of research questions demand different types of evidence, and relying on a single search backend would systematically bias the agent’s coverage.

From a harness design perspective, two principles guide the search infrastructure. The first is resilience through layered fallbacks. The primary web search tool queries a search engine API to return structured results including titles, URLs, and snippets. When the API is unavailable or rate-limited, the tool automatically falls through to a browser-based search backend, and further to a lightweight HTML scraping fallback that requires no API key at all. This three-tier fallback chain ensures that the agent can always perform searches regardless of deployment constraints, preventing a single point of failure in the information acquisition layer from halting the entire research process.

The second principle is transparency to the model. The system supports an optional headless browser backend for pages that require JavaScript rendering, authentication, or anti-bot bypass. Crucially, the browser is not exposed as a separate tool; it functions as a transparent fallback within the existing search and retrieval tools. The model does not know or need to know whether a particular result was obtained through an API call, a headless browser session, or an HTML scraping fallback. This abstraction keeps the tool interface minimal and prevents the model from needing to reason about infrastructure-level routing decisions, allowing it to focus entirely on research strategy. The browser backend supports two distinct contexts: a headless stealth context used as a fallback search engine, and a context that connects to the user’s real browser via a debugging protocol, allowing the agent to access pages behind authentication walls using the user’s existing login sessions.

3.4 Read (Pre-Injection Content Compression)

The most fundamental distinction between a coding agent and a deep search agent lies in the nature of the content they process. A coding agent operates on source code files that are typically compact, well-structured, and can be injected into the context window in their raw form without significant overhead. In contrast, a deep search agent must process web pages that routinely span 50,000 to 100,000 characters each, much of which consists of navigation menus, advertisements, and other content irrelevant to the research query. Injecting raw web content into the context window is extremely expensive: fetching as few as five pages could exhaust the entire context budget before the model has an opportunity to synthesize across sources. This fundamental mismatch between the large volume of web content and the limited context window capacity necessitates a compression strategy that sits between retrieval and reasoning.

SearchClaw addresses this through a multi-stage content processing pipeline that every fetched page traverses before entering the main context window. As shown in Figure 1, the pipeline proceeds as follows. In the **first** stage, the page is retrieved through a tiered fetching strategy that prioritizes reliability and content quality. The system attempts a series of retrieval methods in order of capability: a web reading API that handles JavaScript rendering, PDFs, and complex layouts (e.g., Jina Reader); a headless browser backend capable of executing dynamic content (e.g., Playwright); and direct HTTP retrieval with local HTML-to-markdown conversion as a final resort. This fallback chain ensures robust content acquisition across diverse web environments without depending on any single service. In the **second**

stage, the retrieved content undergoes a size check against a configurable threshold. Content below this threshold is considered sufficiently compact and enters the context in its raw form, avoiding unnecessary processing overhead. Content exceeding the threshold proceeds through the remaining stages. In the **third** stage, regardless of size, the full content is persisted to a disk cache before any compression takes place. This unconditional caching is a critical design choice that preserves the complete original content for potential later access. In the **fourth** stage, a lightweight side-query model performs query-aware extraction: the user’s original research question is provided alongside the page content, and the model extracts key facts, relevant URLs, structural navigation links, and notable quotes while discarding boilerplate and irrelevant material. This query-aware design ensures that the extraction is tailored to the current research context rather than producing a generic summary. The resulting extraction typically compresses the original content to 3,000 to 5,000 characters, achieving a 70 to 90 percent reduction in context consumption. In the **final** stage, the compressed extraction enters the main context along with metadata referencing the cached full content. If the side-query extraction fails at any point, the system falls back to raw truncation at a configurable character limit, preserving system availability at the cost of extraction precision.

The unconditional disk caching in the third stage enables what we term the “*deep-read*” pattern, a two-tier access model that mirrors how human researchers interact with documents. The fast path delivers compressed key facts into the context immediately, allowing the model to quickly assess a source’s relevance and extract essential information. The deep path allows the model to return to specific sections of the cached full content by specifying a heading name, keyword query, or line range through a dedicated deep-read tool. This pattern decouples the initial assessment of a source from detailed examination: the model can skim many sources through the fast path, then selectively deep-read only the sections that warrant closer investigation. The deep-read tool includes path traversal protection that verifies all requested file paths fall within the designated cache directory, guarding against prompt injection attacks that might attempt to trick the agent into reading arbitrary filesystem locations.

3.5 Citation Management

Citations serve as the evidentiary backbone of a research agent’s output, connecting each factual claim to its verifiable source. In the deep research setting, citation management presents a unique challenge: the agent consults numerous sources across multiple search iterations, and the final response must accurately attribute information to the correct origins. A naive approach that relies solely on the model to remember and cite sources from earlier in the conversation is fragile, particularly after context compaction has removed the original tool results.

SearchClaw implements a two-tier citation system that combines automatic discovery with explicit registration. The first tier operates passively: every search and retrieval tool automatically creates citation records from the content it returns, capturing URLs, page titles, and representative snippets as the agent works. These records accumulate on the session state regardless of whether the model explicitly references them, forming a broad pool of all sources the agent has encountered during the research process. This automatic tracking ensures that no source is lost even if the model neglects to cite it in its final response.

The second tier requires active participation from the model. Through a dedicated citation tool, the agent can formally register a citation that directly supports a specific claim in its answer. Each explicit citation includes the source URL, title, a relevant supporting snippet, the source type (e.g., web, academic, or news), and an optional relevance note explaining why the source matters to the current argument. Explicitly registered citations are distinguished from automatically discovered ones in the final output, allowing users to identify not only which sources the agent encountered but which ones it judged to be directly relevant to its conclusions.

This two-tier design addresses two complementary failure modes. Automatic discovery prevents information loss: even if the model forgets to cite a source or context compaction removes the original retrieval record, the citation pool retains a complete provenance trail. Explicit registration prevents citation dilution: rather than attributing the response to every page the agent happened to visit, the curated citations tie specific claims to specific evidence, producing a more useful and trustworthy bibliography. The accumulated citation records also serve as the evaluation substrate for the quality gate system described in Section 3.6, where hooks assess citation count and source diversity to determine whether the agent has gathered sufficient evidence before finalization.

3.6 Hook-Based Quality Gates

Left unchecked, LLM-based research agents exhibit a strong tendency toward premature termination: the model generates a superficially plausible answer after consulting only one or two sources, without verifying claims across multiple references or ensuring that all aspects of the query have been adequately addressed. This behavior is particularly problematic in the deep research setting, where answer quality is inherently multi-dimensional, spanning factual grounding, source diversity, topical coverage, and response thoroughness. Unlike coding tasks where output correctness can be deterministically verified through compilation or test execution, research quality lacks a single binary signal, making it tempting for the model to declare completion prematurely.

SearchClaw addresses this through a hook-based quality gate system that interposes between the model’s intent to finalize and the actual delivery of the response. The system is organized around lifecycle events, with the primary event being “stop”, which is triggered whenever the model produces a response without requesting any tool calls, signaling its intent to conclude the research. At each stop event, a set of pluggable hook functions evaluate the current session state against configurable quality criteria. Each hook inspects the accumulated evidence, citations, research plan status, or response properties, and returns a pass or fail verdict along with optional feedback text. When all hooks pass, the response is finalized and delivered. When any hook fails, its feedback is injected into the conversation as a targeted message that identifies the specific deficiency, and the loop continues. This creates a self-correcting dynamic where the model receives actionable guidance on exactly what is missing, rather than a generic instruction to “do more research”.

The system includes four built-in hooks that collectively enforce minimum quality standards across complementary dimensions. (1) The **citation quality hook** requires a configurable minimum number of explicitly registered citations before finalization is permitted, ensuring that the response is grounded in verifiable sources. When the model attempts to answer without having invoked any tools at all, likely relying on injected memory content or parametric knowledge, the feedback is specifically tailored to demand source verification rather than generic additional searching, avoiding wasted turns on redundant retrieval. (2) The **source diversity hook** requires citations from a minimum number of distinct website domains, preventing the model from citing a single source repeatedly and declaring the research complete. (3) The **answer completeness hook** enforces a minimum response length, catching degenerate outputs such as “I couldn’t find information about this”. (4) The **plan completeness hook**, which operates in conjunction with the structured research planning module described in Section 3.2, blocks finalization while any sub-tasks in the research plan remain incomplete.

Two additional design decisions are worth noting. First, all hooks adopt a fail-open error handling strategy: if a hook’s evaluation itself throws an exception due to implementation bugs or unexpected state, the hook is skipped rather than blocking answer delivery indefinitely. This ensures that the quality enforcement layer improves the system’s output without ever becoming a source of system failure. Second, all citation-related hooks share a non-research query classifier that determines whether the current query genuinely requires web research. Conversational queries such as greetings or capability questions are exempted from citation and completeness requirements. This classifier uses a lightweight LLM side-query rather than brittle pattern matching, and its results are cached within each evaluation cycle to prevent redundant calls when multiple hooks check the same classification.

3.7 Context Management

Even with the pre-injection content compression described in Section 3.4, long research sessions inevitably accumulate substantial context. Each iteration of the agentic loop appends the model’s reasoning, tool invocation records, and tool results to the conversation history. After ten or more tool invocations, the conversation can approach the model’s context window limit, at which point the system must either halt prematurely or compress the accumulated context to continue productive research. A robust context management strategy is therefore essential to support the extended, multi-turn investigations that characterize deep research tasks.

SearchClaw employs a two-phase compaction strategy that activates when the estimated token count exceeds a configurable threshold. The two phases represent a deliberate trade-off between speed and compression quality, with the system attempting the cheaper operation first and escalating only when necessary.

The first phase, which we call **microcompaction**, operates without any LLM call, making it fast and cost-free. It preserves two anchors of the conversation: the first user message, which contains the original research query, and the most recent messages, which carry the model’s current reasoning state. All older tool-result messages whose

content exceeds a minimum size threshold are fully cleared and replaced with a marker indicating that the content has been removed. Assistant messages are never cleared for two reasons: they are typically short, consisting of a few hundred characters of reasoning, and they often contain tool invocation blocks that subsequent tool-result messages reference by identifier. Clearing them would break the structural contract expected by the model’s API. A key design choice in this phase is the preference for full clearing over truncation. Truncating a tool result to a fixed character count produces misleading partial content: the model observes an incomplete snippet and may hallucinate the missing portion as if it were factual. Full clearing is safer because the model can distinguish between information that is absent and information that is present, and it knows to re-search if the removed content is needed.

The second phase, **full compaction**, is triggered only when microcompaction fails to reduce the context below the threshold. This phase invokes a side-query model to produce a comprehensive structured summary of the entire conversation. The summary follows a prescribed format that preserves the original research question, all key findings organized by topic, every source URL encountered during the session, contradictions identified between sources, all user messages including any clarifications, and the current state of the research plan. Source URLs receive special emphasis in the summarization instructions because they constitute the evidentiary backbone of the final response and must never be lost during compression. The summarizer uses an internal analysis scratchpad to organize its reasoning before producing the final summary; this scratchpad is stripped from the output as it serves no informational purpose once the summary is written. After full compaction, the conversation is reduced to three messages: the original user query, the structured summary as an assistant message, and the most recent assistant message if it is distinct from the summary. This minimal representation provides the model with the original question, a complete record of accumulated findings, and its most recent reasoning context.

A fallback mechanism provides resilience against summarization failures. If the full compaction side-query fails due to model errors or timeout, the system reverts to an aggressive variant of microcompaction that retains only the last two messages, sacrificing research history to maintain system availability. This layered fallback design, from microcompaction to full compaction to aggressive microcompaction, ensures that context management never becomes a point of system failure, even at the cost of degraded research continuity in edge cases.

3.8 Memory

The preceding harness components operate within the scope of a single research session. However, real-world research is often an ongoing process where users return with related queries over days or weeks, gradually refining their understanding of a topic. Without a mechanism for carrying knowledge across sessions, the agent treats every interaction as a fresh start, unable to recall which sources proved reliable, what preferences the user expressed, or what reference material was previously discovered. This results in redundant searches, repeated mistakes, and a failure to adapt to the user over time.

SearchClaw addresses this through a persistent memory system that extracts, stores, and retrieves learned knowledge across independent research sessions. The system organizes memories into four categories, each capturing a distinct type of cross-session knowledge: (1) **User profile memories** record the user’s background, expertise level, and research interests, enabling the agent to calibrate the depth and terminology of its responses. (2) **Feedback memories** capture corrections and preferences the user has expressed regarding search behavior, source selection, or quality expectations. (3) **Source reputation memories** track which sources proved reliable or unreliable for particular topics and which domains to prefer or avoid in future research. (4) **Reference memories** preserve bookmarked sources, useful URLs, key facts, and databases discovered during past research that may be valuable for related future queries.

The memory lifecycle consists of two key processes: **post-session extraction** and **pre-query retrieval**. After each completed research session, an extraction process analyzes the session’s trajectory, including the original query, the final response, and the research plan’s accumulated findings, to identify zero to three noteworthy learnings worth persisting. This extraction is LLM-driven: a side-query evaluates the session response and determines whether any new knowledge merits long-term storage. Existing memory headers are included in the extraction prompt to prevent duplicate entries. The extraction runs asynchronously after the response has been delivered to the user, ensuring that it never adds latency to the research experience. Before each new research session, a retrieval process selects relevant memories to inject into the model’s system prompt. Rather than relying on keyword matching, the retrieval employs an LLM-based selector: all memory headers are loaded as lightweight metadata, and a side-query model identifies which memories are most relevant to the current query. Full content is loaded only for selected memories, up to a configurable maximum. This LLM-driven retrieval ensures semantic relevance rather than superficial lexical overlap.

Three design principles govern the memory retrieval process to prevent it from degrading rather than enhancing research quality. First, the selector always runs, even when the total memory count is small, ensuring that irrelevant memories are never injected simply because they exist. Second, the selector is explicitly permitted to return an empty list when no stored memories are relevant to the current query. Third, the system fails to empty on errors: if the retrieval process encounters an exception, it returns no memories rather than falling back to recent entries, preventing potentially irrelevant content from contaminating the research context. These principles reflect a broader design philosophy that inaccurate or irrelevant context is more harmful than absent context, a principle that also motivates the full-clear strategy in context compaction (Section 3.7) and the fail-open design in quality gates (Section 3.6).

A final safeguard addresses the risk of memory over-reliance. When memories are injected into the system prompt, they carry an explicit annotation stating that they are contextual hints from past sessions and not citable sources. The agent is instructed to search for and verify information independently even when memories appear to directly answer the current query. This constraint is reinforced by the citation quality hook (Section 3.6), which detects when the model attempts to finalize a response without having invoked any research tools and injects feedback demanding source verification. Together, these mechanisms ensure that memories serve as useful priors that accelerate and inform research, without becoming a shortcut that bypasses the evidence-gathering process.

4 Conclusion

This paper presented SearchClaw, a deep research agent that explores the application of harness engineering to the deep web search domain. By focusing engineering effort on the control infrastructure surrounding a single powerful foundation model, including structured research planning, pre-injection content compression, hook-based quality gates, and persistent cross-session memory, the system demonstrates that principled harness design offers a viable path for building capable search agents without the cost of model training or the complexity of multi-agent orchestration. We observed that transferring harness patterns from coding to deep search is not straightforward, as the noise, volume, and uncontrollability of web content require domain-specific mechanisms that go beyond what coding harnesses provide. We hope this early exploration encourages the community to consider harness engineering as a research direction for LLM-based agents beyond the coding domain, and we plan to extend this work with systematic benchmark evaluations and cross-model analyses in future versions of this report.

References

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, Hao Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, Tao Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nat.*, 645(8081):633–638, 2025. doi: 10.1038/S41586-025-09422-Z. <https://doi.org/10.1038/s41586-025-09422-z>.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and

- leverage search engines with reinforcement learning. *CoRR*, abs/2503.09516, 2025. doi: 10.48550/ARXIV.2503.09516. <https://doi.org/10.48550/arXiv.2503.09516>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *CoRR*, abs/2603.28052, 2026. doi: 10.48550/ARXIV.2603.28052. <https://doi.org/10.48550/arXiv.2603.28052>.
- Baixuan Li, Bo Zhang, Dingchu Zhang, Fei Huang, Guangyu Li, Guoxin Chen, Huifeng Yin, Jialong Wu, Jingren Zhou, Kuan Li, Liangcai Su, Litu Ou, Liwen Zhang, Pengjun Xie, Rui Ye, Wenbiao Yin, Xinmiao Yu, Xinyu Wang, Xixi Wu, Xuanzhong Chen, Yida Zhao, Zhen Zhang, Zhengwei Tao, Zhongwang Zhang, Zile Qiao, Chenxi Wang, Donglei Yu, Gang Fu, Haiyang Shen, Jiayin Yang, Jun Lin, Junkai Zhang, Kui Zeng, Li Yang, Hailong Yin, Maojia Song, Ming Yan, Peng Xia, Qian Xiao, Rui Min, Ruixue Ding, Runnan Fang, Shaowei Chen, Shen Huang, Shihang Wang, Shihao Cai, Weizhou Shen, Xiaobin Wang, Xin Guan, Xinyu Geng, Yingcheng Shi, Yuning Wu, Zhuo Chen, Zijian Li, and Yong Jiang. Tongyi deepresearch technical report. *CoRR*, abs/2510.24701, 2025a. doi: 10.48550/ARXIV.2510.24701. <https://doi.org/10.48550/arXiv.2510.24701>.
- Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, Weizhou Shen, Junkai Zhang, Dingchu Zhang, Xixi Wu, Yong Jiang, Ming Yan, Pengjun Xie, Fei Huang, and Jingren Zhou. Websailor: Navigating super-human reasoning for web agent. *CoRR*, abs/2507.02592, 2025b. doi: 10.48550/ARXIV.2507.02592. <https://doi.org/10.48550/arXiv.2507.02592>.
- Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models. *CoRR*, abs/2501.05366, 2025c. doi: 10.48550/ARXIV.2501.05366. <https://doi.org/10.48550/arXiv.2501.05366>.
- Xiaoxi Li, Jiajie Jin, Guanting Dong, Hongjin Qian, Yutao Zhu, Yongkang Wu, Ji-Rong Wen, and Zhicheng Dou. Webthinker: Empowering large reasoning models with deep research capability. *CoRR*, abs/2504.21776, 2025d. doi: 10.48550/ARXIV.2504.21776. <https://doi.org/10.48550/arXiv.2504.21776>.
- Ryan Lopopolo. Harness engineering: leveraging codex in an agent-first world, 2026. <https://openai.com/index/harness-engineering/>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. <https://openreview.net/forum?id=dHng2O0Jjr>.
- Prithvi Rajasekaran. Harness design for long-running application development, 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing wikipedia-like articles from scratch with large language models. In Kevin Duh, Helena Gómez-Adorno, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 6252–6278. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.NAACL-LONG.347. <https://doi.org/10.18653/v1/2024.naacl-long.347>.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving AI tasks with chatgpt and its friends in hugging face. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. http://papers.nips.cc/paper_files/paper/2023/hash/77c3e6a367922d003ff102ffb92b658-Abstract-Conference.html.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *CoRR*, abs/2503.05592, 2025. doi: 10.48550/ARXIV.2503.05592. <https://doi.org/10.48550/arXiv.2503.05592>.
- Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. Zerosearch: Incentivize the search capability of llms without searching. *CoRR*, abs/2505.04588, 2025. doi: 10.48550/ARXIV.2505.04588. <https://doi.org/10.48550/arXiv.2505.04588>.

- Zhengwei Tao, Jialong Wu, Wenbiao Yin, Junkai Zhang, Baixuan Li, Haiyang Shen, Kuan Li, Liwen Zhang, Xinyu Wang, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Webshaper: Agentially data synthesizing via information-seeking formalization. *CoRR*, abs/2507.15061, 2025. doi: 10.48550/ARXIV.2507.15061. <https://doi.org/10.48550/arXiv.2507.15061>.
- Jialong Wu, Baixuan Li, Runnan Fang, Wenbiao Yin, Liwen Zhang, Zhengwei Tao, Dingchu Zhang, Zekun Xi, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Webdancer: Towards autonomous information seeking agency. *CoRR*, abs/2505.22648, 2025a. doi: 10.48550/ARXIV.2505.22648. <https://doi.org/10.48550/arXiv.2505.22648>.
- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. Webwalker: Benchmarking llms in web traversal. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 10290–10305. Association for Computational Linguistics, 2025b. <https://aclanthology.org/2025.acl-long.508/>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. https://openreview.net/forum?id=WE_vluYUL-X.
- Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 414–431. Association for Computational Linguistics, 2025. doi: 10.18653/V1/2025.EMNLP-MAIN.22. <https://doi.org/10.18653/v1/2025.emnlp-main.22>.